

QIntern 2026 Project Proposal

Quantum Reservoir Memory for Continuous-Time Temporal Graph Learning

A hybrid quantum temporal graph network for dynamic quantum-network prediction

Field	Entry
Project title	Quantum Reservoir Memory for Continuous-Time Temporal Graph Learning
Short name	Hybrid Quantum Reservoir Temporal Graph Network (H-QTGN)
Mentor(s)	Primary mentor: Krishna Bhatia. Co-mentor(s): Shalini Devendrababu.
Project duration	01 July - 15 August 2026
Expected team size	3-5 interns, depending on selected skill mix
Project mode	Remote, collaborative research-and-implementation project
Keywords	Quantum reservoir computing; temporal graph networks; dynamic graphs; quantum networks; QML; event-stream prediction
Primary output	Open-source prototype, reproducible experiments, dataset/simulator pipeline, final technical report, and mini-workshop presentation

Form-ready note. The QIntern form can use either this uploaded PDF link or a shorter markdown version. This document contains the required title, mentor names, objectives, QIntern-duration work plan, and deliverables in one self-contained proposal.

Hybrid Quantum Reservoir Temporal Graph Network (H-QTGN)

QIntern implementation focuses on a scalable hybrid split: quantum reservoir for temporal features, classical memory for graph state.

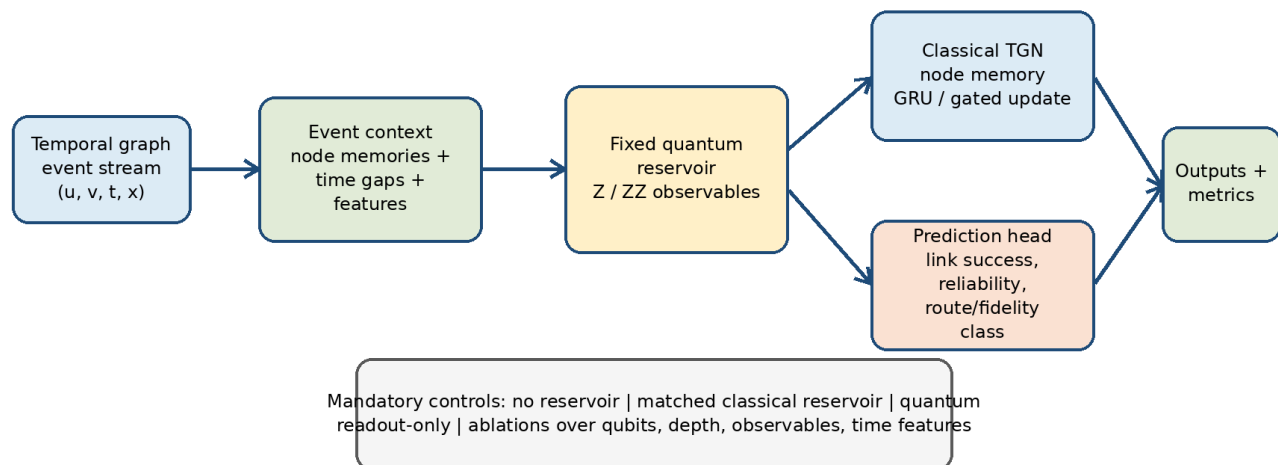


Figure 1. Proposed H-QTGN architecture. The quantum reservoir is used for temporal feature extraction; graph memory, training, evaluation, and prediction remain classical for scalability and reproducibility.

1. Project summary

This project asks whether a small fixed quantum reservoir can improve temporal event prediction in dynamic graphs when embedded inside a scalable temporal graph network. The proposed model, H-QTGN, keeps persistent node memories and graph indexing classical, while a quantum reservoir produces nonlinear temporal features from timestamped interaction events. The project is designed as a QIntern-sized implementation and research study: interns will build a clean prototype, compare it to classical temporal graph and classical reservoir baselines, and evaluate it on dynamic quantum-network prediction tasks and, if time permits, one public temporal graph benchmark.

The central application is dynamic quantum-network operations. A quantum communication network naturally forms a temporal graph: nodes are quantum processors, repeaters, routers, or memories; edges are entanglement-generation attempts, link uses, route requests, or channel-state updates; event features include time gaps, fidelity estimates, memory pressure, congestion, and success or failure. A model that predicts link success, fidelity threshold satisfaction, route quality, or early failure can support scheduling, routing, and anomaly detection in simulated quantum networks.

2. Motivation and background

Temporal graph learning handles graphs whose edges, features, and interactions evolve over time. Continuous-time temporal graph models process events such as (source, destination, time, features) in chronological order and maintain memory states for nodes. Temporal Graph Networks (TGNs) are especially relevant because they expose modular components: message construction, memory update, embedding computation, and task prediction.

Quantum reservoir computing (QRC) uses a fixed quantum dynamical system as a nonlinear temporal feature generator. In contrast to fully trainable quantum neural networks, the internal reservoir can remain fixed while only the classical readout and surrounding model are trained. This makes QRC attractive for near-term simulation and small-device experimentation.

The research gap is the intersection of three areas: event-driven temporal graph learning, reservoir computing for temporal dynamics, and quantum reservoir computing. Existing quantum graph neural networks usually focus on static or weakly temporal graph settings, while existing QRC work usually focuses on generic time series rather than graph event streams. H-QTGN targets this gap by placing a quantum reservoir exactly where it is most appropriate: inside the temporal component of a continuous-time graph learner.

3. Objectives

- Define a clear hybrid quantum temporal graph architecture in which a quantum reservoir handles temporal event features while persistent node memories remain classical.
- Implement a reproducible event-driven H-QTGN prototype with a frozen quantum reservoir, a classical memory updater, and prediction heads for binary and regression tasks.
- Build or adapt a dynamic quantum-network event dataset using a simulator or a documented surrogate generator.
- Compare H-QTGN against mandatory controls: no-reservoir temporal graph baseline, matched classical random reservoir, quantum readout-only baseline, and simple recency/MLP baselines.
- Analyze whether any observed benefit comes from quantum reservoir dynamics rather than dimensionality, random features, time-gap leakage, or parameter count.
- Deliver open-source code, reproducible scripts, a final report, and a concise presentation for the QIntern mini-workshop.

4. Research questions

- RQ1: Can a fixed quantum reservoir provide useful nonlinear temporal features for event-driven graph prediction?
- RQ2: Does H-QTGN outperform a matched classical reservoir when reservoir output dimension, temporal split, and classical memory size are controlled?
- RQ3: Is the gain, if any, stronger in quantum-network event streams where the underlying system is quantum-adjacent?
- RQ4: Which reservoir design choices matter most: qubit count, circuit depth, observable family, partial reset/fading memory, or input encoding?
- RQ5: Are quantum reservoir features more useful for ranking metrics, calibration, or robustness under temporal distribution shift?

5. Technical approach

5.1 Data model

The temporal graph is represented as a chronological event stream:

$$e_k = (u_k, v_k, t_k, x_k, y_k)$$

Here u_k and v_k are endpoint nodes, t_k is the timestamp, x_k is an event-feature vector, and y_k is the supervised target when available. Each node i maintains a classical memory vector $m_i(t)$. The memory is updated only when the node participates in an event, preserving the asynchronous structure of TGNs.

5.2 H-QTGN event-processing loop

1. Read the next temporal event in chronological order.
2. Retrieve current memories for the two endpoint nodes.
3. Construct an event context vector from event features, endpoint memories, time gaps, and optional structural summaries.
4. Encode the context vector into a small fixed quantum reservoir using angle encoding or data re-uploading.
5. Measure reservoir observables such as single-qubit Z expectations and selected ZZ correlators.
6. Fuse the measured reservoir vector with event features and endpoint memories.
7. Update endpoint memories using a classical GRU or gated residual block.
8. Predict event success, link reliability, route quality, or a task-specific label.

5.3 Quantum reservoir design

The first implementation should use a frozen random circuit or fixed Ising-style reservoir with $q = 4-8$ qubits and shallow depth. A classical preprocessing layer maps the event context vector to rotation angles. The reservoir state evolves through fixed entangling gates and input re-uploading layers, after which observables are measured to produce r_k . The frozen-reservoir design keeps training stable and makes the comparison against a matched classical reservoir clean.

5.4 Why this is not a fully quantum TGN

A fully quantum TGN would require quantum storage and retrieval of node memories, quantum graph-neighborhood access, and asynchronous node-specific updates. That design is conceptually interesting but not suitable for a short remote internship. H-QTGN instead uses quantum dynamics only for temporal nonlinear feature generation, while scalable indexed memory and evaluation remain classical.

6. Primary use-case: dynamic quantum-network prediction

The flagship use-case is a simulated quantum communication network. Each event describes an attempted operation or link-use event. The model predicts whether the operation will succeed, whether a fidelity threshold will be met, or which route/class is likely to be reliable. This application is a strong fit because the data are naturally temporal, graph-structured, and quantum-adjacent.

Component	Project definition
Nodes	Quantum processors, repeaters, memories, routers, end users
Edges/events	Entanglement attempts, link updates, route requests, memory allocations, swap operations
Event features	Time gap, link length, loss/noise estimate, memory load, prior success rate, queue length, fidelity proxy
Targets	Success/failure, fidelity class, route viability, failure/anomaly flag, expected latency class
Operational value	Better routing, scheduling, resource allocation, and early-warning signals in dynamic quantum networks

7. Baselines and ablations

The project must avoid a weak-baseline comparison. The minimum comparison set is:

- TGN-like no-reservoir baseline: event features, time gaps, classical node memory, and classical decoder.
- Matched classical reservoir: same output dimension as the quantum reservoir, using a fixed random tanh or echo-state-style feature map.
- Quantum readout-only baseline: classical TGN features followed by a quantum circuit only at the final prediction stage.
- MLP/logistic event-feature baseline: uses event features and time gaps without graph memory.
- Recency heuristic: predicts from recent success/failure history and time-gap statistics.

Core ablations:

- qubit count $q = 2, 4, 6, 8$ where feasible;
- circuit depth and entangling pattern;
- single-qubit Z versus $Z + ZZ$ observables;
- with and without time-gap features;
- with and without partial reservoir reset/fading-memory control;
- reservoir output dimension matched across quantum and classical variants;
- multiple random seeds for reservoir initialization and model training.

8. Evaluation metrics

- Binary classification: AUROC, AUPRC, F1, log loss, Brier score, and expected calibration error.
- Regression: MAE, RMSE, R^2 , and calibration/coverage if uncertainty estimates are included.
- Temporal robustness: train on earlier events and test on later shifted periods; avoid random event shuffling.
- Systems metrics: inference time per event, reservoir feature-computation cost, and memory footprint.
- Reproducibility: at least three random seeds during QIntern; five seeds as a stretch goal if compute allows.

9. Work plan for 01 July - 15 August 2026

Dates	Milestone	Expected activities
01-05 Jul	Onboarding and scope freeze	Read TGN/QRC primers, set up GitHub repository, define coding standards, finalize primary task and dataset/simulator path.
06-12 Jul	Data and baseline pipeline	Implement temporal event loader, surrogate or simulator-derived quantum-network dataset, chronological split, simple baselines, metric scripts.
13-19 Jul	Quantum reservoir module	Implement frozen quantum reservoir, input encoding, observable extraction, caching options, and matched classical reservoir control.
20-26 Jul	H-QTGN integration	Connect event encoder, reservoir features, classical memory store, GRU/gated updater, and task head. Run smoke tests end-to-end.
27 Jul-02 Aug	Main experiments	Run H-QTGN, no-reservoir, classical reservoir, readout-only, and simple baselines under identical temporal splits.
03-09 Aug	Ablations and robustness	Run qubit/depth/observable/time-feature ablations, calibration plots, temporal-shift tests, and runtime profiling.
10-15 Aug	Finalization and mini-workshop package	Freeze code, prepare final report, prepare slides/demo notebook, record limitations, and package reproducibility instructions.

10. Deliverables

- D1: Clean GitHub repository with installation instructions, environment file, and documented module structure.
- D2: Temporal graph dataset pipeline for dynamic quantum-network event prediction, including data schema and generation/simulation notes.
- D3: Implemented H-QTGN model with frozen quantum reservoir and classical node memory.
- D4: Mandatory baselines and ablation scripts, including matched classical reservoir control.
- D5: Reproducible experiment logs, metric tables, and plots generated from saved predictions.
- D6: Final technical report describing method, experiments, limitations, and future work.
- D7: Mini-workshop presentation/demo notebook suitable for QIntern outcome presentation.

11. Intern roles and skill requirements

Role	Responsibilities
------	------------------

Temporal graph / ML intern	Implements event loader, memory store, baselines, metrics, and TGN-like components.
Quantum reservoir intern	Implements quantum reservoir simulation, input encoding, observables, and classical-reservoir controls.
Quantum-network simulation intern	Builds or adapts a dynamic quantum-network event generator/simulator and validates labels/features.
Reproducibility and analysis intern	Maintains experiment scripts, plots, seed logs, documentation, and final report integration.

Helpful prerequisites: Python, PyTorch or NumPy, basic machine learning, basic graph concepts, and introductory quantum computing. Interns do not need to know all areas in advance; the project is structured so that each student can specialize and collaborate.

12. Risks and mitigation

Risk	Mitigation
Quantum reservoir does not outperform matched classical reservoir	Report honestly; analyze calibration, robustness, and task regimes. The result remains useful as a controlled boundary study.
Simulation or quantum-network data is delayed	Use a documented surrogate generator first; treat simulator integration as a stretch goal.
Quantum simulation is too slow	Limit to 4-6 qubits, cache reservoir features, reduce event count, and prioritize matched baselines.
Scope becomes too broad	Freeze minimal architecture by mid-July and move advanced variants to future work.
Intern skill mismatch	Split tasks into ML, quantum, simulation, and reproducibility tracks with weekly integration checkpoints.

13. Stretch goals

- Add a graph-structured quantum reservoir whose connectivity reflects hardware topology or a compressed network topology.
- Run a small quantum hardware demonstration for a toy subset if access and shot budget permit.
- Evaluate on one public temporal graph benchmark such as Wikipedia, Reddit, MOOC, or LastFM using chronological splits.
- Implement a partially trainable input encoder for the reservoir and compare it with the frozen-reservoir version.
- Package the dynamic quantum-network event dataset as a reusable benchmark with clear metadata.

14. Selected references

- [1] Rossi, E. et al. Temporal Graph Networks for Deep Learning on Dynamic Graphs. arXiv:2006.10637, 2020.
- [2] Xu, D. et al. Inductive Representation Learning on Temporal Graphs. arXiv:2002.07962, 2020.
- [3] Kumar, S., Zhang, X., and Leskovec, J. Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks. arXiv:1908.01207, 2019.
- [4] Trivedi, R. et al. DyRep: Learning Representations over Dynamic Graphs. arXiv:1803.04051, 2019.
- [5] Fujii, K. and Nakajima, K. Harnessing Disordered-Ensemble Quantum Dynamics for Machine Learning. Physical Review Applied, 2017.
- [6] Nakajima, K. and Fischer, I. Reservoir Computing: Theory, Physical Implementations, and Applications. Springer, 2021. See chapters on quantum reservoir computing.
- [7] Verdon, G. et al. Quantum Graph Neural Networks. arXiv:1909.12264, 2019.
- [8] Wehner, S., Elkouss, D., and Hanson, R. Quantum Internet: A Vision for the Road Ahead. Science, 2018.
- [9] Coopmans, T. et al. NetSquid, a discrete-event simulation platform for quantum networks. Communications Physics, 2021.
- [10] Wu, X. et al. SeQUeNCe: a customizable discrete-event simulator of quantum networks. Quantum Science and Technology, 2021.
- [11] Huang, S. et al. Temporal Graph Benchmark for Machine Learning on Temporal Graphs. arXiv:2307.01026, 2023.